

EXAMEN BLANC COMPLET — SPRING BOOT / JPA / REST

Durée : 2h

Licence : ISI — JEE & Spring Boot

PARTIE 1 — ENTITÉS JPA

1. Créer les entités :

- Department
- Session
- Trainer

Utiliser :

- @Entity
- Lombok
- relations JPA correctes

2. Transformer la relation ManyToMany entre Trainer et Session en utilisant une entité intermédiaire Trainee

Utiliser :

- @EmbeddedId
- @MapsId
- @ManyToOne

3. Créer la clé composite TrainerSessionId.

PARTIE 2 — REPOSITORIES

4. Créer SessionRepository.

a) Retourner les sessions ayant une durée supérieure à une valeur donnée.

b) Retourner les sessions dont le titre contient un mot.

c) Retourner les sessions appartenant à un département donné.

d) Retourner les sessions enseignées par un formateur donné.

e) Retourner :

- le nom du département
- le nombre total de sessions

Utiliser :

- group by
- count

f) Retourner :

- le titre de la session
- la somme totale des heures consommées

PARTIE 3 — REPOSITORY AVANCÉ

5. Créer TrainerRepository.

a) Retourner les trainers ayant une spécialité donnée.

b) Retourner :

- le nom du trainer
- le nombre total de sessions enseignées

c) Retourner les trainers qui enseignent plus de 2 sessions.

Utiliser :

- having

d) Retourner :

- nom trainer
- titre session
- date affectation

PARTIE 4 — SERVICES

6. Créer :
- SessionService
- TrainerService

avec les méthodes :
- add
- findAll
- findById

7. Dans TrainerSessionService, créer une méthode permettant :
- d'ajouter un trainer
- d'ajouter une session
- de créer leur association

PARTIE 5 — REST CONTROLLER

8. Créer un SessionRestController.

a) Ajouter une session
POST /sessions

b) Afficher toutes les sessions
GET /sessions

c) Afficher une session par id
GET /sessions/{id}

Utiliser :
- Optional
- ResponseEntity

d) Supprimer une session
DELETE /sessions/{id}

PARTIE 6 — CONTROLLER CLASSIQUE

9. Créer une méthode permettant :
- d'ajouter un trainer
- récupérer une session existante
- associer trainer et session

Afficher :
System.out.println(...)

PARTIE 7 — HÉRITAGE

10. Créer les classes :
- User
- Admin
- Student

Utiliser :
- @Inheritance(strategy = InheritanceType.JOINED)
- @DiscriminatorColumn
- @DiscriminatorValue

11. Créer une requête permettant de retourner uniquement les admins.

PARTIE 8 — QUESTIONS THÉORIQUES

12. Différence entre @Controller et @RestController.

13. À quoi sert mappedBy ?

14. Différence entre @OneToMany et @ManyToMany.

15. Rôle de @Autowired.
16. Pourquoi utilise-t-on @JsonIgnore ?
17. À quoi sert @MapsId ?
18. Différence entre LAZY et EAGER.

PARTIE 9 — QUESTIONS PIÈGES

Piège 1 :
@OneToMany
private Department department;

Piège 2 :
where s.trainers.name=:name

Piège 3 :
@Query("select t from Trainer where t.name=:name")

Piège 4 :
@OneToMany
private List<Session> sessions;

Piège 5 :
public class TrainerSessionId {
 private Trainer trainer;
 private Session session;
}

Piège 6 :
@Query("select s.department.name from Session s")

Piège 7 :
where s.department=:name

Piège 8 :
@PostMapping

Piège 9 :
return optional.get();

Piège 10 :
@ManyToMany(mappedBy="trainer")

PARTIE BONUS

19. Créer une API REST permettant de retourner :
 - le nom du trainer
 - le nombre total d'heurestrié par ordre décroissant.
 20. Créer une requête permettant de retourner :
 - les sessions qui n'ont aucun trainer
- Utiliser :
 - is empty